



Malgorzata Debska
CS 230 Project Software Design Template
Version 1.0

Table of Contents

CS 230 Project Software Design Template	1
Table of Contents	2
Document Revision History	2
Executive Summary	3
Requirements	3
Design Constraints	3
System Architecture View	3
Domain Model	3
Evaluation	4
Recommendations	5

Document Revision History

Version	Date	Author	Comments
1.0	07/20/2024	Malgorzata Debska	Initial draft of the software design document

Instructions

Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The Gaming Room has partnered with Creative Technology Solutions (CTS) to create a web-based version of their popular Android game, "Draw It or Lose It." This game, similar to the 1980s show "Win, Lose or Draw," involves teams guessing images from a library of stock drawings within a set time. The new web-based version will support multiple platforms, increasing accessibility and reach.

This document details the software design for the new application, focusing on key requirements such as supporting multiple teams with unique names for games, teams, and players. Additionally, only one game instance should be active at a time, and the game must allow real-time interaction with progressively rendered images and timed guessing.

The solution involves developing a distributed web-based environment using modern web technologies to ensure scalability, responsiveness, and efficient game state management. Understanding design constraints like platform compatibility, concurrency management, and real-time interaction is crucial for the project's success. Addressing these through careful design and technology selection will ensure a seamless and engaging user experience across multiple platforms.

Requirements

The Gaming Room has specified a set of business and technical requirements for developing the web-based version of "Draw It or Lose It." These requirements are crucial to ensure the application meets user needs and functions effectively across multiple platforms.

Business Requirements:

- 1. Support for Multiple Teams:** The game must accommodate multiple teams, with each team comprising several players.
- 2. Unique Naming Convention:** The names for games, teams, and players must be unique to avoid conflicts and ensure smooth operation.
- 3. Single Active Game Instance:** There should only be one active instance of the game in memory at any given time to maintain data integrity and prevent duplication.

Technical Requirements:

- 1. Real-Time Interaction:** The game must support real-time interaction, with images progressively rendered and teams making guesses within set time limits.
- 2. Cross-Platform Compatibility:** The web application must be compatible with various web browsers and devices, featuring a responsive design that adjusts to different screen sizes.
- 3. Concurrency Control:** Efficient management of game state and concurrency is necessary to handle multiple users interacting with the game simultaneously.
- 4. Unique Identifiers:** The system must generate and enforce unique identifiers for games,

teams, and players to ensure only one instance of each is active at any time.

5. Scalability: The application must be scalable to accommodate an increasing number of users and game sessions without compromising performance.

Following to these requirements is vital for delivering a robust, scalable, and user-friendly web-based game, thereby expanding the accessibility and reach of "Draw It or Lose It."

Design Constraints

Developing the web-based version of "Draw It or Lose It" involves several key design constraints that must be addressed for successful implementation in a distributed environment. These constraints significantly impact the application development process.

1. Platform Compatibility: The application must work across various web browsers and devices, requiring a responsive design that adapts to different screen sizes and resolutions. Extensive testing across platforms is necessary to ensure consistent performance and user experience.

2. Real-Time Interaction: The game requires real-time interaction with progressively rendered images and timed guesses. This necessitates efficient data handling and low-latency communication technologies to maintain seamless and responsive gameplay.

3. Concurrency Management: The game must handle multiple users and teams simultaneously, requiring robust concurrency control and scalable architecture to manage concurrent sessions without conflicts or performance issues.

4. Unique Identifiers: Ensuring unique game, team, and player names requires a reliable system for generating and validating identifiers. This adds complexity, necessitating a centralized database or service to enforce uniqueness.

5. Single Game Instance: Only one instance of the game should be active at any time, requiring careful management of game state and memory to prevent duplication and ensure data integrity.

6. Scalability: The application must scale to accommodate increasing users and game sessions, requiring scalable technologies like cloud-based services and distributed databases to handle growing demand without compromising performance.

Addressing these constraints through careful planning and appropriate technologies is crucial for developing a robust, scalable, and user-friendly web-based game that meets The Gaming Room's requirements.

System Architecture View

For comprehensive project planning, detailing the system architecture is crucial. Here's an overview of the key components and their interactions:

System Architecture:

- **Presentation Layer:** Handles the user interface, ensuring responsive design for various devices.
- **Application Layer:** Manages game logic, real-time interactions, and game states, using technologies like WebSockets for efficient communication.
- **Data Layer:** Stores game data and manages unique identifiers, utilizing a scalable cloud-based database.

Logical Topology:

- **Client-Side:** Users interact via web browsers, connecting to the presentation layer.
- **Server-Side:** Application servers process client requests and maintain game states.
- **Database:** Centralized storage for data, ensuring real-time access and synchronization.

Domain Model

The provided UML class diagram represents a gaming room application, showcasing several classes and their relationships. At the core of this design is the Entity class, which serves as a base class for Player, Team, and Game. The Entity class encapsulates common attributes id and name, and provides methods for accessing these attributes (getId(), getName()) as well as a toString() method for representing the entity as a string. This structure ensures that the common functionality is centralized, promoting code reuse and maintainability.

The Player class inherits from Entity and adds no additional attributes, but it has a constructor and a toString() method tailored to represent player-specific data. Similarly, the Team class inherits from Entity but adds an attribute players, which is a list of Player objects, indicating a one-to-many relationship. The Team class provides methods for adding players (addPlayer(name: String): Player) and representing the team as a string (toString()), demonstrating encapsulation by managing the collection of players internally.

The Game class, also inheriting from Entity, includes an attribute teams, a list of Team objects, indicating another one-to-many relationship. This class allows for the addition of teams (addTeam(name: String): Team) and provides a toString() method to represent the game as a string. By encapsulating teams within a game, the design ensures that the management of teams is centralized within the Game class, promoting a clear hierarchy and modular design.

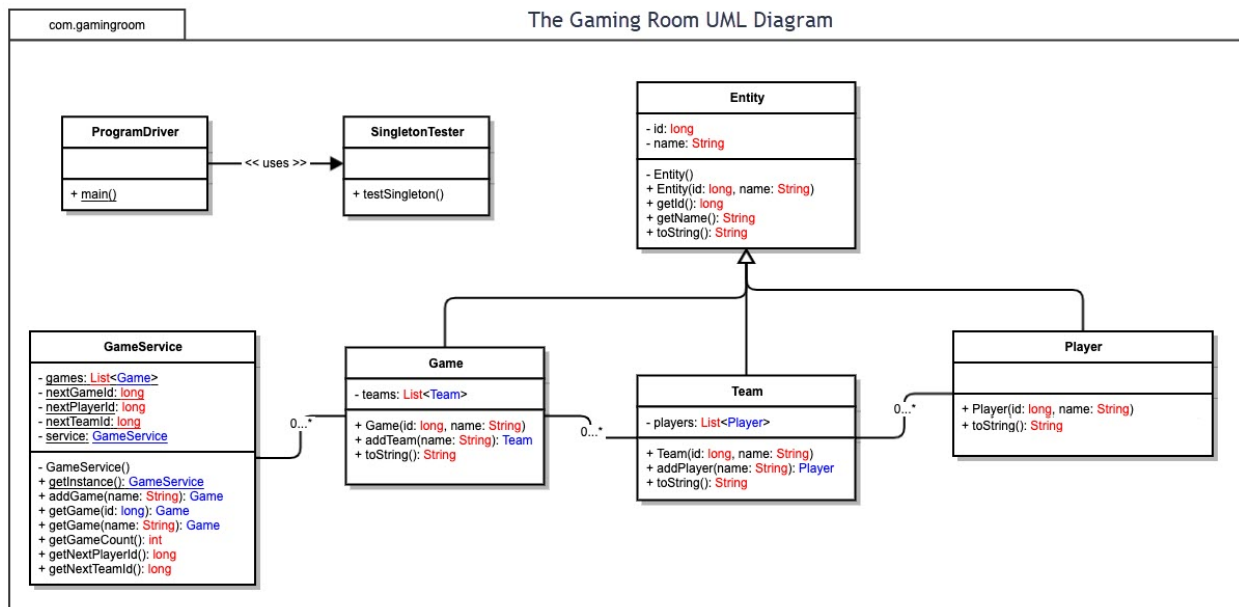
The GameService class plays a crucial role in managing the overall application. It follows the Singleton pattern, ensuring that only one instance of GameService exists throughout the application. This is achieved through a private constructor and a static getInstance() method. The GameService class maintains a list of Game objects and provides methods for adding games (addGame(name: String): Game), retrieving games by ID or name (getGame(id: long): Game, getGame(name: String): Game), and obtaining the game count (getGameCount(): int). It also manages unique identifiers for games, players, and teams (nextGameId, nextPlayerId, nextTeamId), ensuring that each entity has a unique ID.

To test the Singleton implementation, the SingletonTester class includes a testSingleton() method, which is used by the ProgramDriver class in its main() method. This setup ensures that the Singleton pattern is correctly implemented and functioning as expected.

The design demonstrates several object-oriented programming principles. Encapsulation is evident as each class manages its own data and exposes only necessary methods for interaction. Inheritance is utilized to create a hierarchy of entities, promoting code reuse. Polymorphism is enabled by the Entity class, allowing Player, Team, and Game to be treated as Entity objects where appropriate. The Single Responsibility Principle is adhered to, with each class having a well-defined purpose. Finally, the Singleton pattern in GameService ensures that game management is centralized, preventing conflicts

and inconsistencies.

In conclusion, this UML class diagram showcases a well-structured design that leverages object-oriented principles to create an efficient, maintainable, and scalable gaming room application. The modular design facilitates future extensions and modifications, ensuring that the system can evolve to meet new requirements while maintaining its integrity and performance.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Mac servers offer stability, seamless integration with Apple products, and robust security. However, they are costly and less common in server environments, which might limit available tools and support.	Linux is popular for its open-source nature, flexibility, and robust performance. It's cost-effective but requires high technical expertise for setup and maintenance.	Windows servers integrate well with Microsoft products and offer user-friendly interfaces. However, they are more expensive due to licensing fees and have a higher vulnerability to certain attacks.	Mobile devices are generally unsuitable for hosting but can be used for lightweight web servers in specific scenarios. They are limited by hardware capabilities and network stability.
Client Side	Developing for Mac requires expertise in macOS-specific technologies like Swift and Objective-C, with higher costs for hardware and software. Apple's optimized development tools can lead to high-quality applications.	Supporting Linux clients involves dealing with varied distributions and ensuring compatibility. It's cost-effective, but the fragmented ecosystem demands significant expertise.	Developing for Windows involves tools like Visual Studio and the .NET framework. While providing extensive support, the development can be costly and time-consuming.	Supporting mobile clients involves managing the diversity of Android and iOS platforms. Development costs are high due to cross-platform requirements, necessitating expertise in frameworks like React Native or Flutter.
Development Tools	Common tools include Xcode, Swift, and Objective-C, offering a powerful but closed development environment that limits flexibility.	Tools like GCC, Eclipse, and NetBeans are commonly used. Linux offers extensive customization but has a steep learning curve requiring proficiency in command-line tools.	Visual Studio and .NET framework are powerful but expensive. Integration of tools is seamless, enhancing productivity.	Tools include Android Studio, Xcode, and cross-platform frameworks. Mobile development demands a deep understanding of mobile-specific issues like touch interfaces and battery management.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. Operating Platform: Linux is the most versatile and cost-effective option for expanding "Draw It or Lose It" across multiple computing environments. It offers flexibility, robust performance, and supports a wide range of hardware architectures, making it suitable for both server-side and client-side deployments. While it requires higher technical expertise for setup and maintenance, its open-source nature significantly reduces costs compared to proprietary systems like Windows or macOS. Additionally, Linux's strong community support and extensive toolsets make it an ideal platform for scalable and secure deployments.

2. Operating Systems Architectures: Linux, the chosen operating platform, utilizes a monolithic kernel architecture. This architecture integrates device drivers, file system management, and system call interfaces within the kernel, leading to efficient performance and security. Linux's modular nature allows for the inclusion or exclusion of components as necessary, offering flexibility and optimization opportunities. The architecture supports various file systems such as ext4, XFS, and Btrfs, ensuring compatibility and adaptability to diverse storage needs. Also, Linux excels in networking capabilities, which are crucial for the real-time interaction and distributed nature of the game.

3. Storage Management: For storage management, I recommend using a combination of local storage for temporary data and cloud-based storage solutions like Amazon S3 or Google Cloud Storage for persistent data. Cloud storage provides scalability, durability, and accessibility, essential for handling the game's data needs. Using a relational database like PostgreSQL or MySQL, both of which are well-supported on Linux, ensures efficient management of game states, user data, and real-time interactions. These databases offer robust transaction management, data integrity, and support for complex queries.

4. Memory Management: Linux employs advanced memory management techniques like paging and segmentation to optimize the use of physical memory. It uses a virtual memory system, allowing each process to have its own address space, thereby preventing processes from interfering with one another. The kernel's memory management subsystem efficiently handles memory allocation and deallocation, using techniques such as demand paging, which loads memory pages only when needed. These memory management techniques ensure that the "Draw It or Lose It" software runs smoothly even under heavy load, with efficient use of available memory and minimal latency.

5. Distributed Systems and Networks: To enable "Draw It or Lose It" to communicate across various platforms, I recommend a distributed software architecture using microservices. This approach allows different components of the game to operate independently, facilitating scalability and maintainability. RESTful APIs or gRPC can be used for communication between

microservices, ensuring interoperability and efficient data exchange. WebSockets can be employed for real-time updates and interactions, providing a seamless gaming experience. The network infrastructure should leverage reliable cloud services like AWS, Google Cloud, or Azure, which offer dependable connectivity, scalability, and redundancy. These cloud platforms provide essential services like load balancing, auto-scaling, and content delivery networks (CDNs) to ensure high availability and performance. Managing dependencies between components involves using service registries and discovery mechanisms, ensuring that services can locate each other dynamically. Addressing connectivity issues and outages involves implementing retry mechanisms, circuit breakers, and fallbacks to maintain resilience and service continuity.

6. Security: Security is critical for protecting user information and ensuring the integrity of the game. Linux provides robust security features, including user and group permissions, SELinux (Security-Enhanced Linux), and AppArmor for access control and policy enforcement. For securing data in transit, using TLS (Transport Layer Security) ensures encrypted communication between clients and servers. For user authentication and authorization, integrating OAuth2.0 or OpenID Connect offers secure and standardized methods for managing user access. Data at rest should be encrypted with strong encryption standards. Regular security audits, vulnerability scanning, and patch management are essential for maintaining a secure environment. Also implementing multi-factor authentication adds an additional layer of security for user accounts. Additionally, utilizing security information and event management (SIEM) tools helps monitor and analyze security events in real-time, enabling quick detection and response to potential threats. These security measures will help The Gaming Room protect user information and ensure the secure operation of "Draw It or Lose It" across various platforms.

References:

Bishop, M. (2018). *Computer security: Art and science* (2nd ed.). Addison-Wesley Professional.

Stallings, W. (2018). *Operating systems: Internals and design principles* (9th ed.). Pearson.

Jones, M. T. (2016). *Anatomy of the Linux kernel*. IBM Developer. Retrieved from <https://developer.ibm.com/articles/l-linux-kernel/>

PROJECT 2

SERVER SIDE- various platforms for their characteristics, advantages, and weaknesses for hosting a web-based software application.

When evaluating platforms for hosting a web-based application that needs to scale to thousands of players, several factors come into play, including server-based deployment methods and potential licensing costs.

Linux emerges as the most suitable platform for hosting a scalable web-based application. It excels in handling high traffic due to its support for various web servers such as Apache, Nginx, and Lighttpd, all of which are optimized for performance and scalability. Linux's open-source nature allows for extensive customization and cost-effective solutions, making it highly adaptable for different needs. However, the setup and maintenance of Linux servers require a high level of technical expertise, which could be a challenge for teams without specialized skills.

Windows is also a strong contender for hosting web-based applications. Windows Server editions are designed to manage high traffic effectively and support web servers like IIS. The platform offers a user-friendly environment with strong integration into Microsoft technologies, providing a comprehensive ecosystem of tools and third-party applications. Despite these advantages, the cost of Windows Server licensing can be significant, including additional fees for client access licenses (CALs) and software assurance. Furthermore, Windows servers may be more vulnerable to certain types of security threats, which could be a concern for applications requiring high security.

Mac is generally less suitable for large-scale deployments. While Mac servers, including macOS Server, provide a stable and secure environment with good integration with Apple products, they are better suited for smaller or development environments rather than high-traffic production scenarios. The higher costs associated with Mac hardware and software, coupled with its limited scalability and less common use in enterprise settings, make it less practical for hosting a web-based application that requires significant scalability.

Mobile devices are not appropriate for hosting web-based applications that need to handle high traffic. Their hardware limitations and network instability prevent them from effectively managing large numbers of concurrent users. Mobile devices might be useful for specific, lightweight tasks, but they cannot meet the demands of a high-traffic, scalable web application. Licensing costs for mobile devices are generally not significant, but their limitations in hosting capabilities render them unsuitable for The Gaming Room's requirements.

To conclude, for hosting a scalable web-based application, Linux and Windows are the most viable options. Linux offers a flexible and cost effective solution with robust performance, while Windows provides powerful capabilities and strong integration with Microsoft services, though at a higher cost. Mac and mobile devices are less suitable due to their higher costs and limitations in scalability and performance.

CLIENT SIDE- the software development considerations (cost, time, expertise) that are necessary for supporting multiple types of clients.

To broaden the client's application from an Android-exclusive platform to include support for iOS, Android, and traditional desktop operating systems, with a responsive HTML interface, several key aspects of software development need to be considered. These include the development process, associated costs, and the expertise required to ensure compatibility across diverse platforms.

For the development process, adopting cross-platform frameworks is essential. Tools like React Native and Flutter streamline the creation of mobile applications by allowing developers to maintain a single codebase for both iOS and Android. On the web side, responsive design frameworks such as Bootstrap or Foundation are instrumental in ensuring that the application adjusts seamlessly to different screen sizes and device orientations. This approach is critical for providing a consistent user experience across desktops, tablets, and smartphones. Techniques like flexible grid layouts and media queries are necessary to ensure that the application performs well on various web browsers and devices, regardless of their differences.

In terms of cost, developing a cross-platform application can be more expensive initially. This is due to the need for specialized knowledge in various frameworks and the complexity involved in integrating and testing the application across multiple platforms. Although cross-platform tools can help minimize the need for separate codebases, the costs of extensive testing, performance optimization, and managing updates for different operating systems and browsers can be considerable. Maintenance costs are also significant, as ongoing updates and support are necessary to keep the application compatible with evolving browser and operating system versions. Additionally, there may be costs associated with licensing development tools and testing services, which can further impact the budget.

Regarding expertise, developers need a broad skill set to address the varied requirements of this project. Proficiency in front-end development is essential, including skills in HTML, CSS, and JavaScript, and familiarity with responsive design frameworks like React, Angular, or Vue.js. For mobile development, experience with cross-platform tools such as React Native or Flutter is crucial for creating applications that work efficiently on both iOS and Android. A deep understanding of testing and performance optimization is also important to ensure the application performs well across different devices and browsers. Effective coordination between front-end and back-end teams is necessary to ensure smooth integration and communication with the server-side infrastructure.

In short, creating an application that supports multiple platforms involves a detailed approach, including the use of cross-platform frameworks, responsive design, and managing associated costs and technical challenges. By investing in the appropriate tools and expertise, and ensuring comprehensive testing and ongoing maintenance, the client can successfully deliver a modern, responsive application that meets user needs across various devices and operating systems.

DEVELOPMENT TOOLS

Developing a cross-platform application that functions smoothly across iOS, Android, and desktop operating systems necessitates careful selection of programming languages and development tools. This choice has considerable implications for the development team and the associated expenses.

For mobile platforms, leveraging frameworks such as React Native and Flutter is key. React Native allows developers to write code in JavaScript and React, creating applications for both iOS and Android from a single codebase. Essential tools for React Native development include IDEs like Visual Studio Code or Atom, along with Expo, which facilitates development and testing. Flutter, developed by Google, uses Dart for building high-performance apps that work on both iOS and Android. Developers using Flutter typically rely on Android Studio or IntelliJ IDEA. For iOS-specific development, Xcode is the primary IDE, supporting Swift and Objective-C and providing necessary tools for building and debugging iOS apps. Android Studio serves as the main IDE for Android development, supporting Java and Kotlin, and offering a comprehensive suite of tools for app creation.

When it comes to desktop platforms, Electron stands out as a versatile framework that enables the creation of cross-platform desktop applications using web technologies such as HTML, CSS, and JavaScript. Tools for Electron development include Visual Studio Code or WebStorm. Alternatively, JavaFX can be used for Java-based desktop applications, with support for Windows, Mac, and Linux. JavaFX development is typically done using IDEs like IntelliJ IDEA or Eclipse. For applications specifically designed for Windows, the .NET framework and C# are commonly employed, with Visual Studio as the primary IDE, providing robust tools for Windows application development.

The implications for development teams are significant. Handling various technologies and tools requires specialized knowledge, which may necessitate multiple teams or experts. For instance, one team may concentrate on mobile development using React Native or Flutter, while another focuses on desktop development with Electron or JavaFX. Effective coordination among these teams is crucial to maintain a consistent user experience and ensure smooth integration across different platforms.

Regarding licensing costs, there is variability based on the tools used. React Native and Flutter are both open source and do not have licensing fees, though some associated tools and plugins might come with costs. Xcode and Android Studio are free, but Xcode requires macOS, which adds to the hardware costs. Visual Studio offers both free and paid versions; the Community edition is available at no cost for individual developers and small teams, while the Professional and Enterprise editions have associated fees. IntelliJ IDEA provides a free Community edition and a paid Ultimate edition with additional features. Electron and JavaFX are open-source, but tools like WebStorm or IntelliJ IDEA's Ultimate edition may require licensing fees.

To sum up, developing a cross-platform application involves selecting appropriate programming languages and tools, each influencing the development process and costs. Using frameworks and tools that support various platforms can simplify development, but managing multiple

technologies and coordinating between specialized teams are essential for creating a unified and effective application.

References:

Kaeo, M. (2020). *Network security: A beginner's guide* (3rd ed.). McGraw-Hill Education.

Haverbeke, M. (2018). *Eloquent JavaScript: A modern introduction to programming* (3rd ed.)

Gill, N., & Griffiths, J. (2020). *CSS Mastery: Advanced web standards solutions*